

# Structured

## SOFTWARE ENGINEERING PROJECT

### PREPARED BY:

Layla Ahmad Aldahami |  
Nada Lafi Alharbi |  
Reem Salem Alharbi |  
Lama Abdullah Alolait |  
Leen Abdulkareem Almutairi |  
Wafaa Abdullah Alluhidan |  
Najd mohammed Aljaloud |  
Shahad Fhahad |

### REPRESENTED TO:

DR. SAMIYAH ALANAZI



# THE CONTENT

|              |                                  |
|--------------|----------------------------------|
| <u>03</u>    | INTRODUCTION                     |
| <u>04</u>    | STAKEHOLDERS                     |
| <u>05-07</u> | FUNCTIONAL REQUIREMENTS          |
| <u>08</u>    | NON-FUNCTIONAL<br>REQUIREMENTS   |
| <u>09-16</u> | REQUIREMENT DOCUMENT             |
| <u>17-20</u> | REQUIREMENT SCENARIOS            |
| <u>21</u>    | ARCHITECTURAL PATTERN            |
| <u>22</u>    | SOFTWARE ARCHITECTURAL<br>DESIGN |
| <u>23</u>    | USE CASE DIAGRAM                 |
| <u>24-27</u> | SEQUENCE DIAGRAM                 |
| <u>28</u>    | ACTIVITY DIAGRAM                 |
| <u>29</u>    | CLASS DIAGRAM                    |



# INTRODUCTION

In this project, we aim to analyze and document the software engineering aspects of an existing application called Structured. Structured is a daily planner and visual calendar app that helps users organize their tasks and manage their time efficiently. The app allows users to plan their day using a timeline-based interface, where tasks are assigned to specific time slots.

The objective of our project is not to build a new system, but to study and understand how a real-world productivity app is structured from a software engineering perspective. This includes identifying stakeholders, documenting system requirements, analyzing its architecture, and modeling various components such as use cases and class diagrams.

The contribution of our work is to apply theoretical knowledge gained in the course to a practical case. By reverse-engineering Structured, we can explore how design patterns, requirement specifications, and UML diagrams apply to existing software. This enhances our understanding of system design and analysis through a real and useful application that is widely used in productivity and task management.



# STAKEHOLDERS

- Regular Users: Includes individuals like students, professionals, and parents who use the app to plan tasks, manage schedules, and improve productivity.
- Product Team: Designers, developers, QA testers, and product managers responsible for building, maintaining, and improving the app.
- Integration Platforms: Third-party services such as Google Calendar, Apple Calendar, iCloud, and Siri which integrate with Structured.
- App Store Providers: Platforms (Apple App Store) that distribute the app, manage reviews, and handle subscriptions.
- Customer Support: Handles bug reports, subscription issues, and user feedback to ensure high satisfaction.



# FUNCTIONAL REQUIREMENTS

1. Users can add tasks by entering a title, selecting a time, and adding an optional description.
  - Explanation: Helps users quickly record and plan tasks throughout the day.
2. Tasks can be edited at any time.
  - Explanation: Allows users to update task details whenever needed.
3. Tasks can be deleted at any time.
  - Explanation: Provides the flexibility to remove tasks that are no longer needed.
4. Users can set reminders for tasks.
  - Explanation: Notifies users in advance, reducing the chance of forgetting important tasks.
5. Tasks are displayed in a visual timeline or daily calendar view.
  - Explanation: Makes it easier to view and manage the day at a glance.
6. The system sends push notifications for upcoming tasks.
  - Explanation: Keeps users informed and on schedule.



# FUNCTIONAL REQUIREMENTS CONT.

7. Notification preferences can be customized.

- Explanation: Allows users to choose how and when they receive alerts.

8. Tasks can be set as recurring (e.g., daily or weekly).

- Explanation: Useful for repeating routines or habits without manual input.

9. Users can mark tasks as completed.

- Explanation: Helps track productivity and maintain focus.

10. The app syncs tasks across devices via iCloud.

- Explanation: Ensures consistency when using the app on multiple Apple devices.

11. Backup and restore is supported through iCloud.

- Explanation: Protects user data in case of device change or reinstall.

12. Users can add color tags or icons to tasks.

- Explanation: Helps visually categorize tasks (e.g., personal, work, study).



# FUNCTIONAL REQUIREMENTS CONT.

13. Tasks can include subtasks or checklists.

- Explanation: Useful for breaking down large tasks into smaller steps.

14. Users can search for tasks by keyword.

- Explanation: Helps locate specific tasks, especially in long task histories.

15. Users can share tasks with others.

- Explanation: Useful for collaborative planning or group schedules.

16. Widgets for quick access to tasks on the home screen.

- Explanation: Lets users view tasks without opening the app.



# NON-FUNCTIONAL REQUIREMENTS

## 1. Performance Requirements

- Response Time: The app should load the main timeline within 2 seconds.
- Notification Timing: Task reminders must be delivered within 5 seconds of the scheduled time.

## 2. Availability

- The app should maintain at least 99.9% uptime and remain functional during regular use.

## 3. Security

- Data Protection: All data must be securely encrypted and synced using iCloud's encryption protocols.

## 4. Usability

- The app should include an onboarding tutorial for new users.
- All interface elements should follow Apple Human Interface Guidelines.

## 5. Reliability

- If the internet connection is lost, the app should retry iCloud sync automatically once it's restored.

## 6. Scalability

- The app must support large task lists and frequent updates without slowing down.

## 7. Portability

- The app should run smoothly across supported Apple platforms (iOS, iPadOS, macOS)





# REQUIREMENTS DOCUMENT

## Requirement 1: Create Task

**Function:** Allow users to create a new task with a title, time, and optional description.

**Description:** The app shall enable users to quickly schedule a new task by specifying essential details such as task title, start time, and optional notes. This is the core feature that helps users structure their day effectively.

### Inputs:

Task Title: The name of the task.

Start Time: The time the task is scheduled to begin.

Optional Notes: Additional description or context for the task.

### Outputs:

Task Display: The task is displayed visually in the daily timeline.

Saved Task Record: The task is stored in local database and optionally synced with iCloud.



# REQUIREMENTS DOCUMENT

## CONT.

### **Requirement 1: Create Task cont .**

#### **Action:**

Display a task creation form with fields like duration, color, and optional repetition.

Validate required fields before enabling task submission (e.g., duration must be selected).

If validation fails, highlight missing fields and display contextual error messages.

Enable “Create Task” button only when all mandatory fields are complete.

On successful validation, save the task to local storage and optionally sync via iCloud (Pro).

Show the new task immediately in the timeline.

#### **Pre-conditions:**

The app is running and the user is on the home timeline screen.

#### **Post-conditions:**

The new task is shown in the timeline and saved successfully.

**Side effects:** None.



# REQUIREMENTS DOCUMENT

## CONT.

### **Requirement 2: Set Task Reminder**

**Function:** Allow users to set reminder notifications for a task before it begins.

**Description:** The app shall allow users to receive a notification at a set time before a task starts.

This helps users stay on schedule and not miss important tasks.

#### **Inputs:**

Task ID: The task to which the reminder is applied.

Reminder Time: How many minutes before task start to notify.

#### **Outputs:**

Reminder Notification: A push or local notification is triggered.

Confirmation: User sees reminder settings saved successfully.

#### **Action:**

Enable reminder toggle within task settings.

Schedule local system notification based on task time and selected offset.

#### **Pre-conditions:**

Notifications must be allowed by the user in system settings.

#### **Post-conditions:**

A reminder is scheduled and will notify before task time.

#### **Side effects:**

Frequent notifications may cause user fatigue.



# REQUIREMENTS DOCUMENT

## CONT.

### **Requirement 3: Mark Task as Completed**

**Function:** Allow users to mark a task as completed with visual feedback.

**Description:** The app shall allow users to tap a task and mark it as done, updating the task's appearance.

This supports habit tracking and visual clarity of task progress.

**Inputs:**

Selected Task: A task already displayed in the timeline.

**Outputs:**

Visual Update: Task appears checked and faded.

State Change: Task is marked as completed in storage.

**Action:**

User selects a task and taps 'Complete'.

App updates task status and refreshes UI.

**Pre-conditions:**

The task exists and is not already completed.

**Post-conditions:**

The task is shown as completed and is no longer editable.

**Side effects:**

Completed tasks must be manually unmarked to edit again.



# REQUIREMENTS DOCUMENT

## CONT.

### **Requirement 4: iCloud Sync (Pro)**

**Function:** Synchronize tasks across Apple devices using iCloud (Pro Feature).

**Description:** The app shall sync task data through iCloud to allow users to access their schedule across devices. It ensures consistency for Pro users who rely on multi-device planning.

#### **Inputs:**

Task Data: Tasks saved locally by the user.

iCloud Credentials: Logged-in Apple ID with sync permissions.

#### **Outputs:**

iCloud Sync Confirmation: Changes reflected on another device.

Remote Data Update: Tasks uploaded/downloaded from the cloud.

#### **Action:**

Enable sync setting (Pro only).

Push local updates to iCloud and fetch remote changes regularly.

#### **Pre-conditions:**

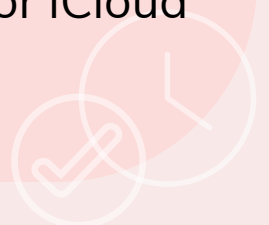
User is subscribed to Pro and signed into iCloud.

#### **Post-conditions:**

All tasks are identical across devices and updated in real-time.

#### **Side effects:**

Sync failures may occur if the internet is unavailable or iCloud is disabled.



# REQUIREMENTS DOCUMENT

## CONT.

### **Requirement 5: Recurring Task (Pro)**

**Function:** Schedule recurring tasks

**Description:** Allows Pro users to create tasks that automatically repeat daily, weekly, or monthly.

**Inputs:**

Task data + repeat configuration (frequency, end date)

**Outputs:**

Recurring tasks generated and added to timeline

**Action:**

User creates or edits a task and enables the 'Repeat' option. The app allows choosing the recurrence pattern (e.g., daily, weekly). The system then automatically generates tasks for future dates based on the set pattern.

**Pre-conditions:**

User has a Pro subscription and recurrence is enabled in settings

**Post-conditions:**

Repeated tasks are shown in the timeline as scheduled instances

**Side effects:**

If not managed, the timeline may become overloaded with recurring tasks



# REQUIREMENTS DOCUMENT

## CONT.

### **Requirement 6: Task Duration and Start Time Handling**

**Function:** Automatically assign default values for task start time and duration if the user does not explicitly provide them.

**Description:**

When creating a task, the user must provide a task title. If no start time or duration is entered, the app shall automatically:  
Assign the next available start time.

Set a default task duration (e.g., 30 minutes).

This ensures that task creation remains fast and user-friendly while maintaining consistent scheduling behavior.

**Inputs:**

Task title (required).

Start time (optional).

Duration (optional; default applied if left empty).

**Outputs:**

A task scheduled with a user-provided title, system-generated start time, and default duration.



# REQUIREMENTS DOCUMENT

## CONT .

### **Requirement 6: Task Duration and Start Time Handling** **CONT .**

#### **Action :**

Display a task creation form with fields for title, start time, and duration .

Require only the title field for submission .

If start time is left empty, assign the next available time slot automatically .

If duration is not modified, apply a default value (e.g., 30 minutes) .

Save the task with the resolved values and display it in the timeline .

#### **Pre-Conditions :**

The task title field must be filled .

Default duration and time logic are defined in the system .

#### **Post-Conditions :**

The task is created with a title, scheduled time, and duration, regardless of user input for time and length .

#### **Side Effects :**

If the user only enters the title and does not review the time and duration, the task may be created with automatic values that do not match their intention .





# REQUIREMENT SCENARIOS

## **1.Scenario: Add Task**

Primary Actor: User

Goal: To add a new task with a title, time, and optional description.

### **Main Success Scenario:**

1. User opens the app.
2. User clicks the “Add Task” button.
3. User enters the task title.
4. User selects the task time.
5. (Optional) User adds a description.
6. User presses the “Create Task” button.
7. The task is displayed in the daily timeline.

### **Alternative Scenarios:**

- 3a. No title is entered → The app suggests task titles based on previous entries.
- 4a. User enters a time that overlaps with another scheduled task → The app accepts the task and displays a visual alert in the timeline indicating that "tasks are overlapping."



# REQUIREMENT SCENARIOS

## CONT.

### **2. Scenario: Set Reminder for Task**

Primary Actor: User

Goal: To set a reminder for an existing task.

#### **Main Success Scenario:**

1. User opens the app.
2. User selects a task from the timeline.
3. User taps the "Edit Task" button.
4. User scrolls down and enables the "Need a reminder?" option.
5. User selects the reminder time (15 minutes before the task).
6. User taps the "Update Task" button.
7. The reminder is successfully scheduled.

#### **Alternative Scenarios:**

- 2a. The selected task no longer exists → The app automatically returns to the home screen without displaying an error message.
- 5a. A reminder time in the past is selected → The app accepts the input, but the reminder may not be triggered at the intended time.



# REQUIREMENT SCENARIOS

## CONT.

### **3.Scenario: Mark Task as Completed**

Primary Actor: User

Goal:To mark a task as completed.

#### **Main Success Scenario:**

1. User opens the app.
2. User navigates to the daily timeline.
3. User taps the task checkbox (circle on the left side of the task).
4. A checkmark (✓) appears, and the task is visually faded to indicate completion.
5. The task remains visible in the timeline as a completed task.

#### **Alternative Scenarios:**

- 3a. User attempts to mark a task that is already completed → No further action occurs, and the task remains unchanged.



# REQUIREMENT SCENARIOS

## CONT.

### **4.Scenario: Sync Tasks via iCloud**

Primary Actor: User

Goal: To synchronize tasks across Apple devices using iCloud.

#### **Main Success Scenario:**

1. User logs into the app on both Device A and Device B.
2. User creates a new task on Device A.
3. The task is automatically synchronized and appears on Device B via iCloud.

#### **Alternative Scenarios:**

- 2a. No internet connection is available → The task is queued for synchronization and will be synced once the connection is restored.
- 3a. iCloud is disabled in device settings → The app displays the message: "Please enable iCloud to sync tasks."



# ARCHITECTURAL PATTERN

We chose the **Layered Architecture** for the Structured app because it's simple, organized, and fits the app's features.

The app is divided into four main layers:

- **Presentation Layer:** Where users interact with the app (timeline, add task, notifications).
- **Application Layer:** Manages screen flow, Pro access, and iCloud permissions.
- **Business Logic Layer:** Handles task creation, reminders, recurring tasks, and rules.
- **Data Layer:** Stores tasks locally and syncs them with iCloud.

This pattern makes the app easier to build, test, and update. Each layer works on its own, so we can fix bugs or add features without affecting everything.

It also matches Apple's development style and supports features like offline mode and cloud sync — which are key parts of Structured.

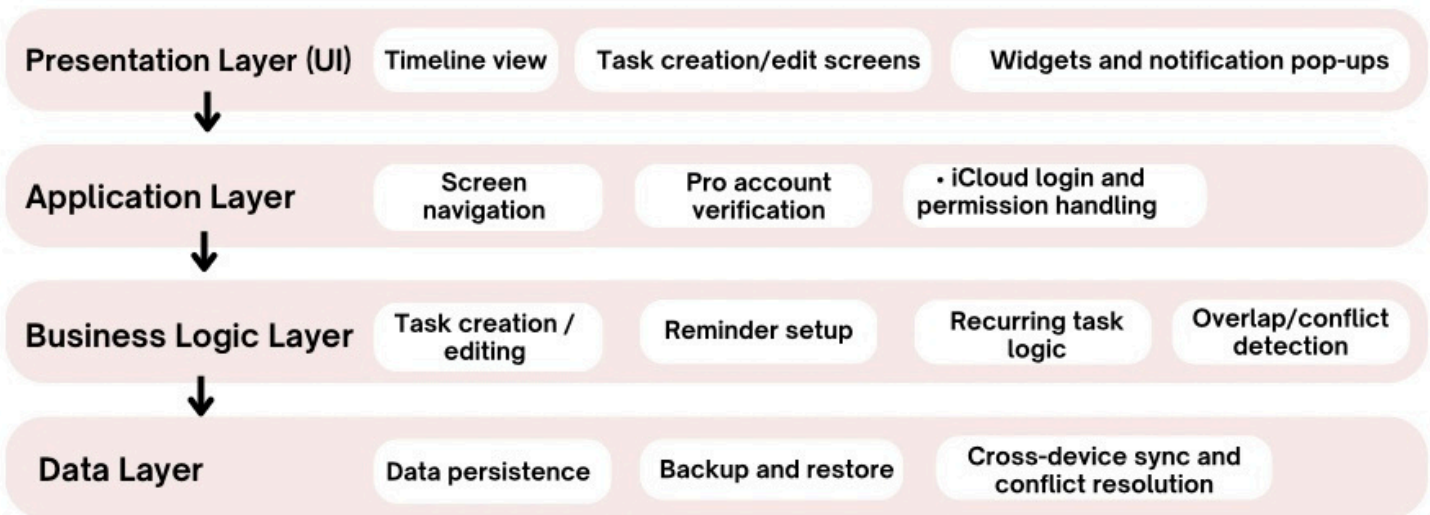
## Why It Fits Structured

- Supports offline functionality with optional cloud sync.
- Promotes clean, modular development for group work.
- Allows smooth feature updates like recurring tasks or widgets.
- Provides a stable and intuitive user experience.

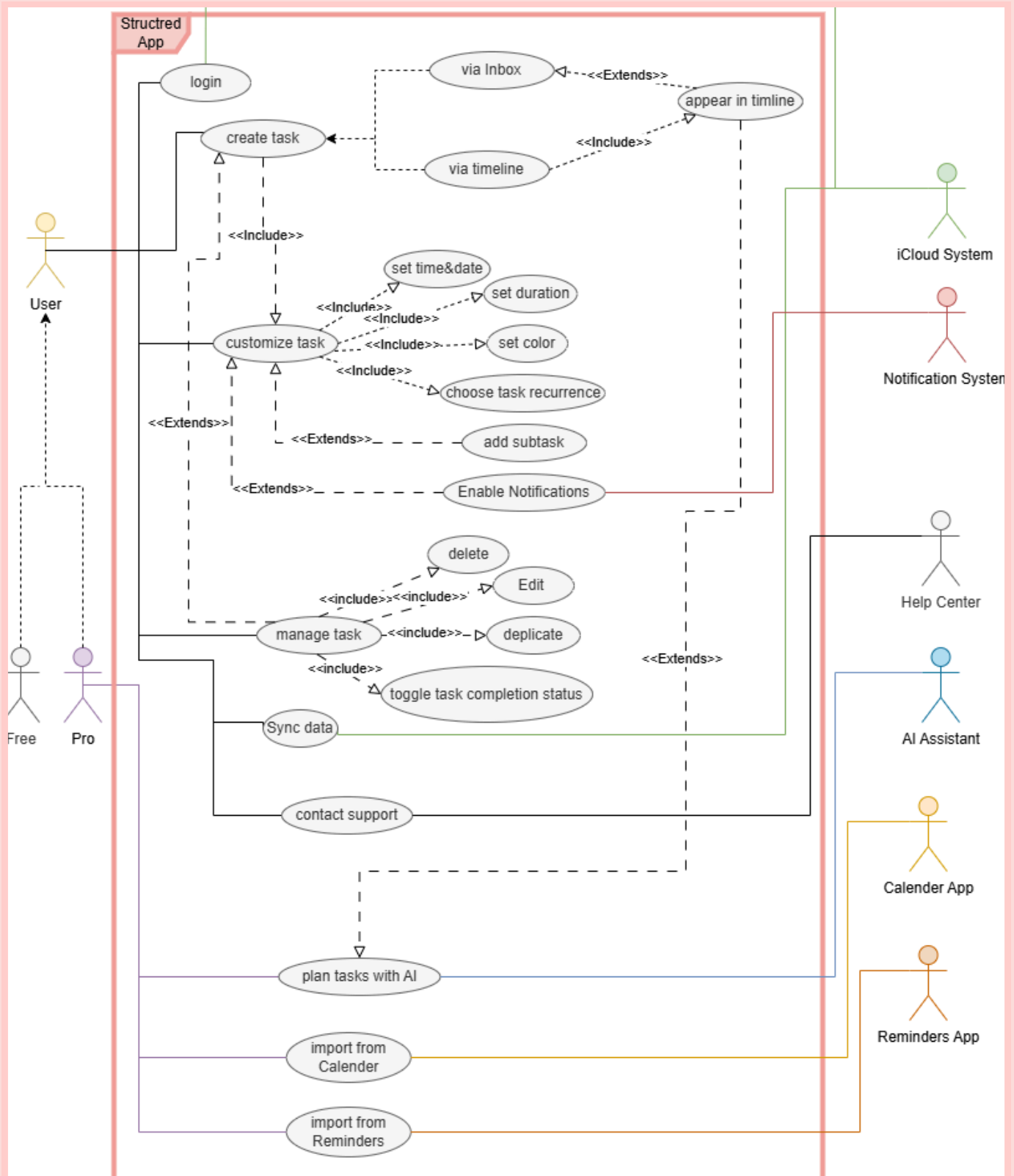


# SOFTWARE ARCHITECTURAL DESIGN

Layered Architecture:



# USE CASE DIAGRAM



[click here! to view more clearly](#)



# SEQUENCE DIAGRAM

- **Add Task**

This diagram shows the process of a leader creating a new task. After clicking "New Task," the input form is submitted to the TaskManager, which stores the task in the database. Upon confirmation, the UI displays a success message to the leader.



[click here! to view more clearly](#)

There's all 3 sequence diagram in this link

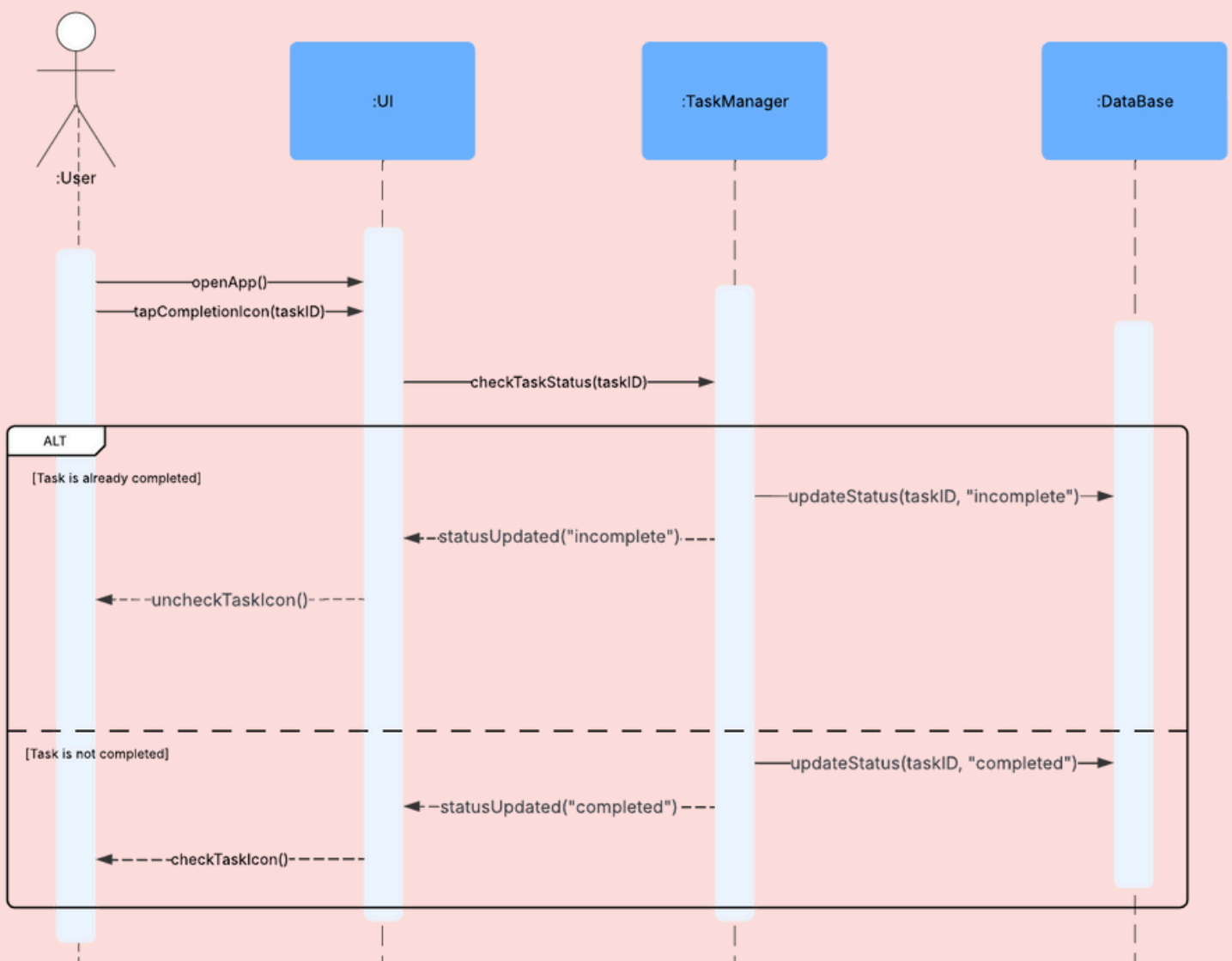




# SEQUENCE DIAGRAM CONT.

- **Mark Task as Completed**

This diagram shows the process of a user toggling the completion status of a task. When the user taps the task's circle icon, the UI checks whether the task is already completed. Based on the current status, the TaskManager updates the task as either completed or incomplete in the database. After confirmation, the UI reflects the new status accordingly.



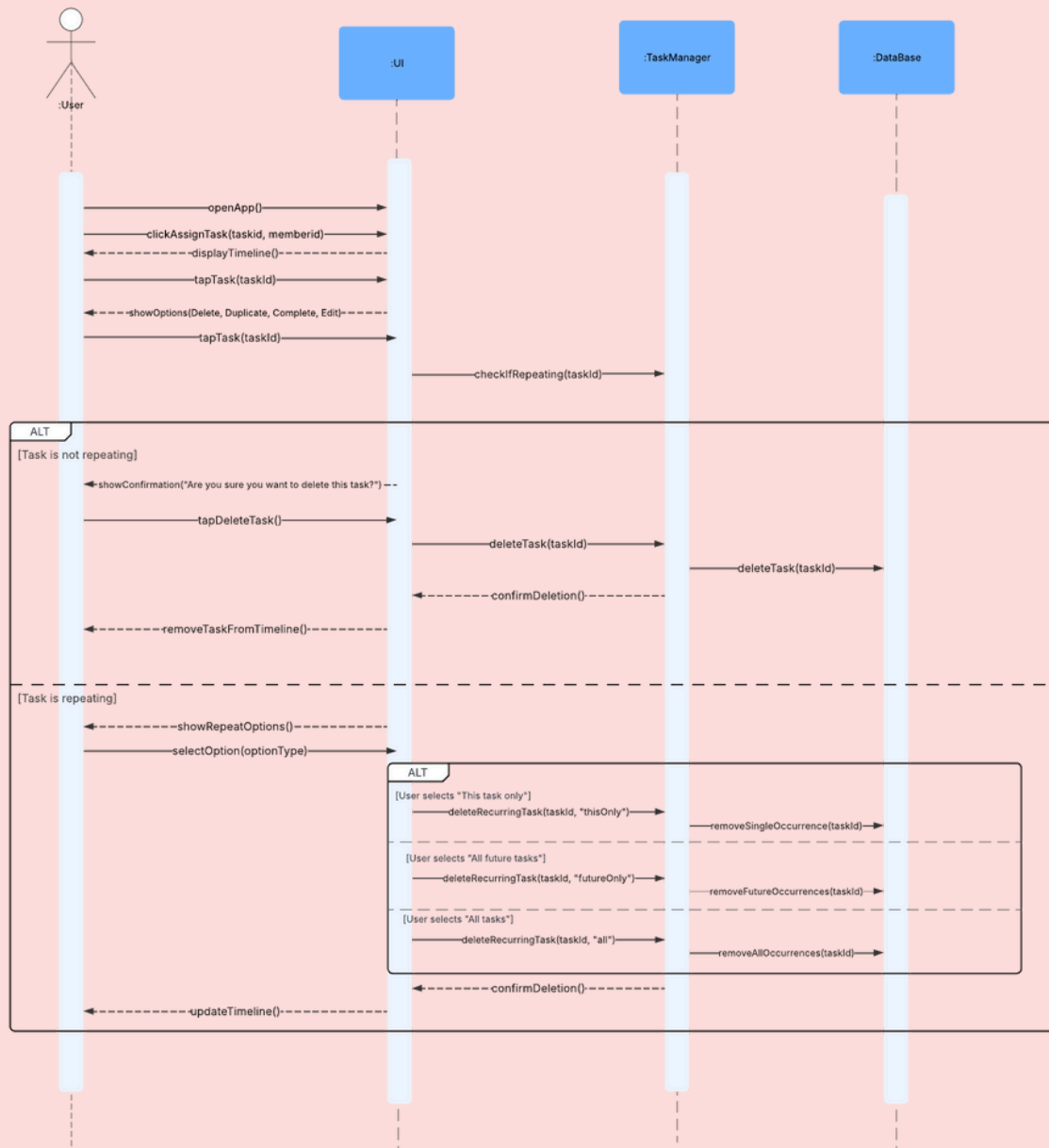
[click here! to view more clearly](#)



# Sequence Diagram Cont.

- **Delete Task**

This diagram shows how a user deletes a task from the Structured app. When the user taps a task and selects “Delete,” the app checks whether the task is a repeating task. If it is not, a simple confirmation dialog is shown, and the task is deleted upon confirmation. If the task is part of a repeating series, the app presents multiple options: delete only this instance, all future tasks, or the entire series. After the user confirms, the TaskManager processes the deletion and updates the timeline accordingly.



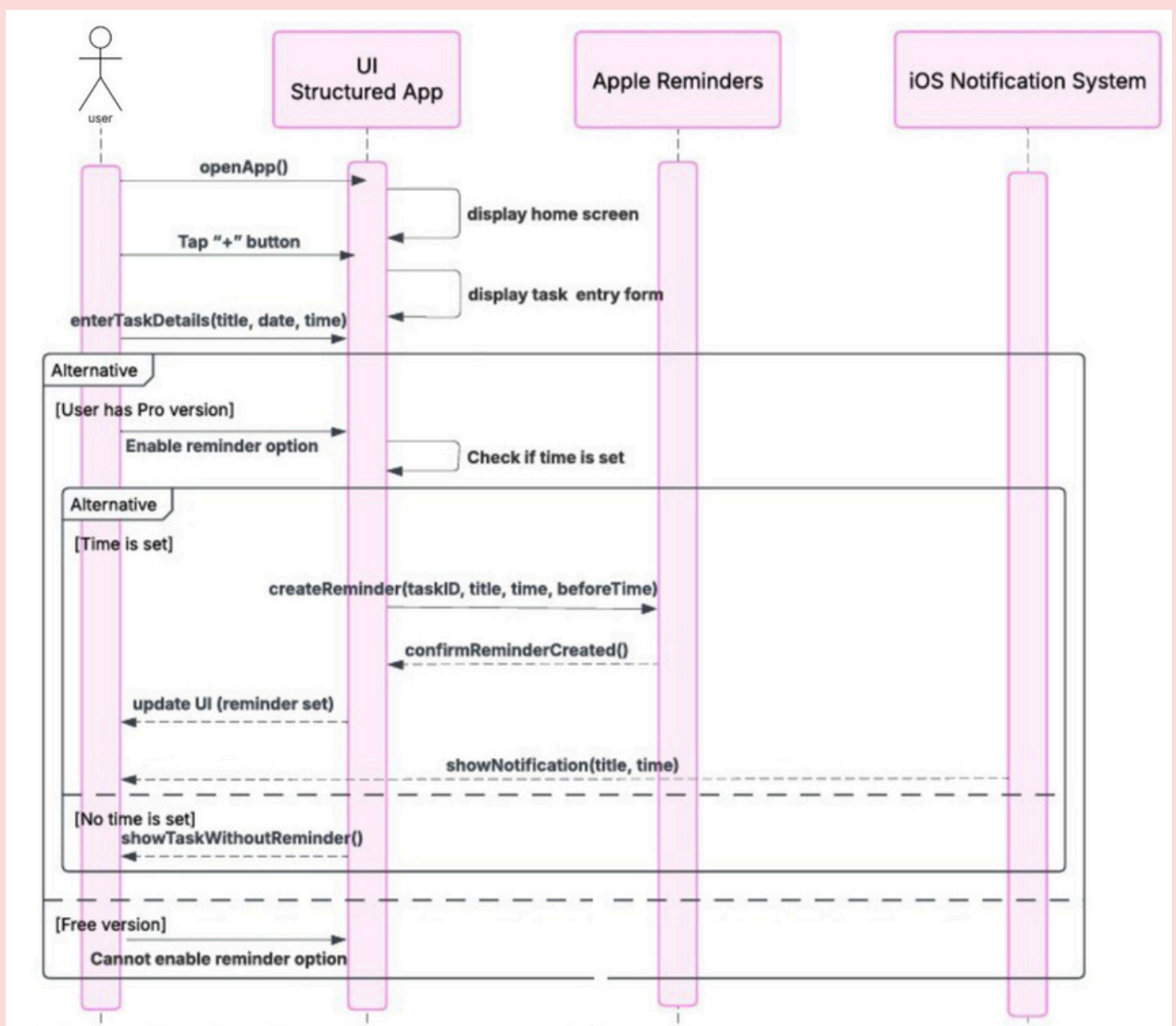
[click here! to view more clearly.](#)



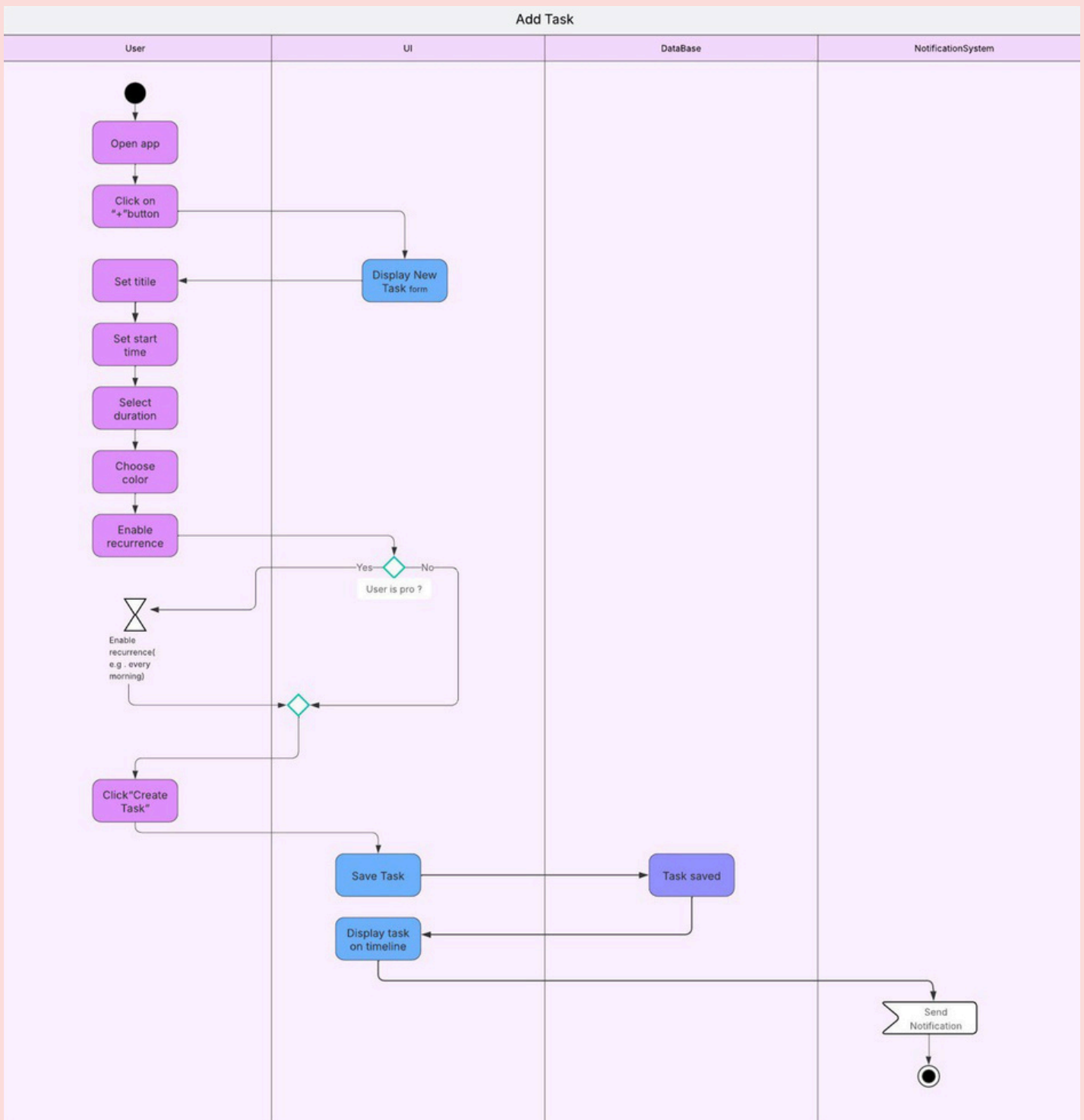
# SEQUENCE DIAGRAM CONT.

- **Set Reminder tasks**

The Set Reminder Task feature in the Structured app allows users to set reminders for their task. If the user has the Pro version, they can enable the reminder option. The app sends the reminder to Apple Reminders, and a notification is sent at the scheduled time. If the user doesn't have the Pro version, they can't use the reminder feature



# ACTIVITY DIAGRAM

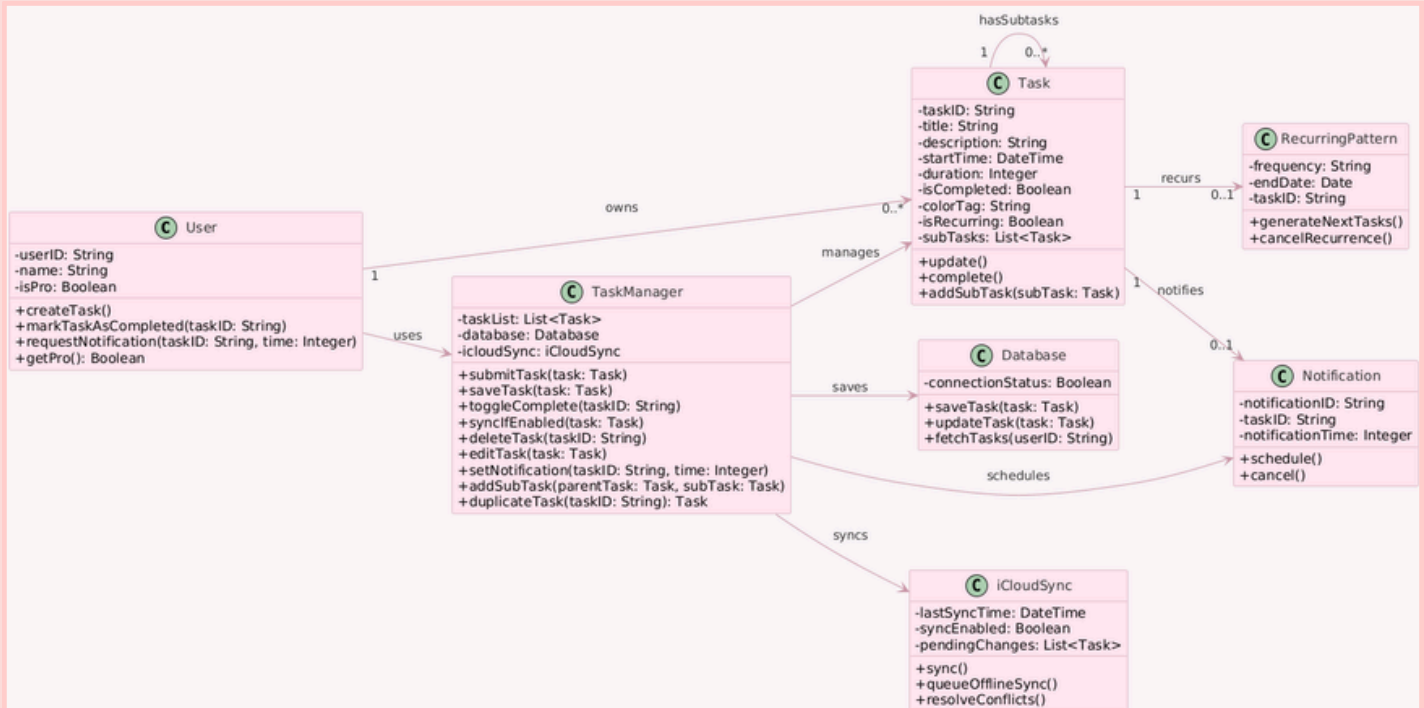


The user opens the app, fills in task details like title, time, duration, and color, and can enable recurrence if they are a Pro user. The task is saved and shown on the timeline, and a notification is sent

[click here!](#) to view more clearly



# CLASS DIAGRAM



[click here! to view more clearly](#)



THE END!

